들어봤니 GraphQL

Google Developer Groups
Sungkonghoe University



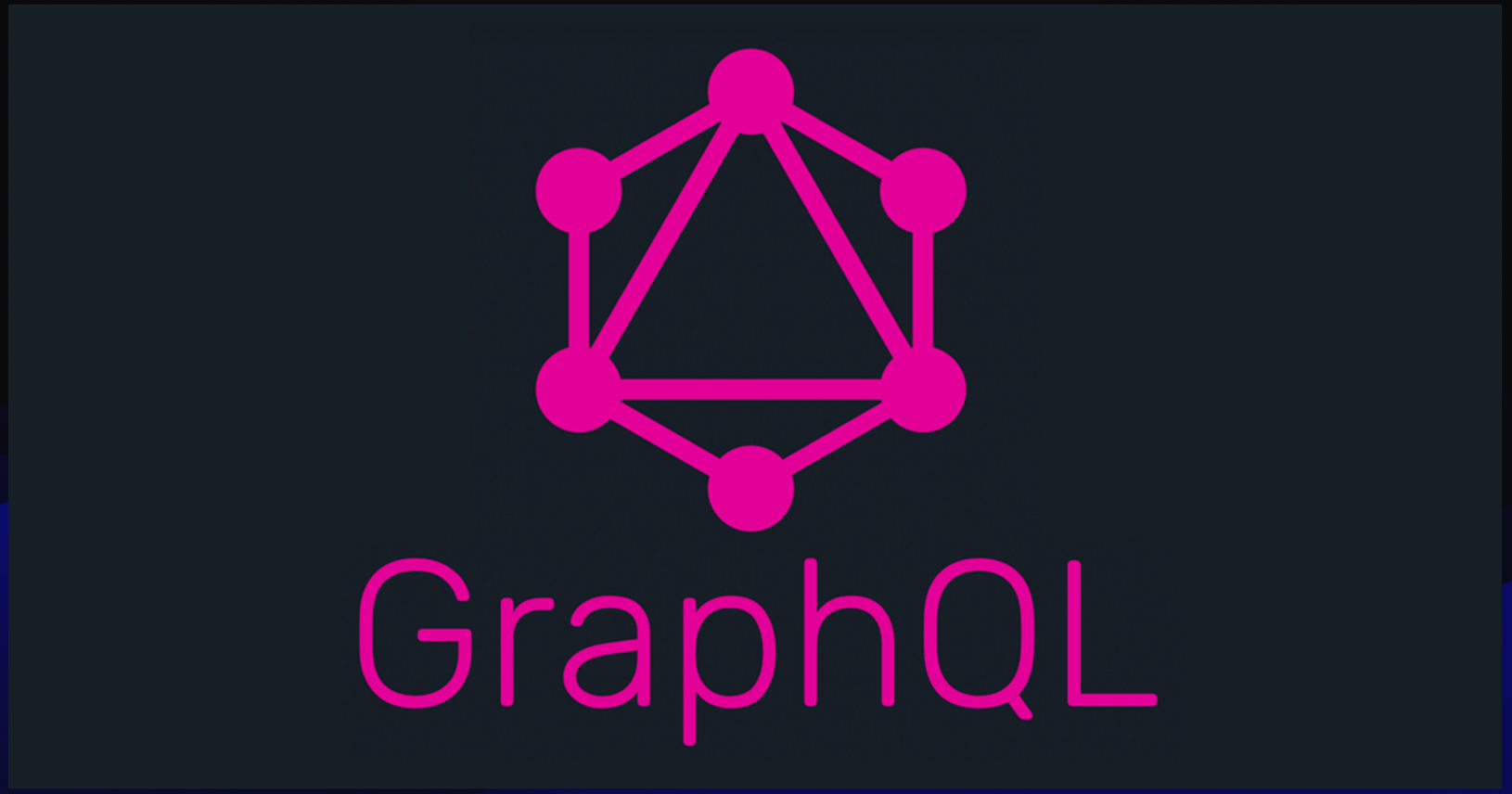
Younha Lee 이윤하
GDG on Campus SKHU 24-25 Organizer



GraphQL? 🤔

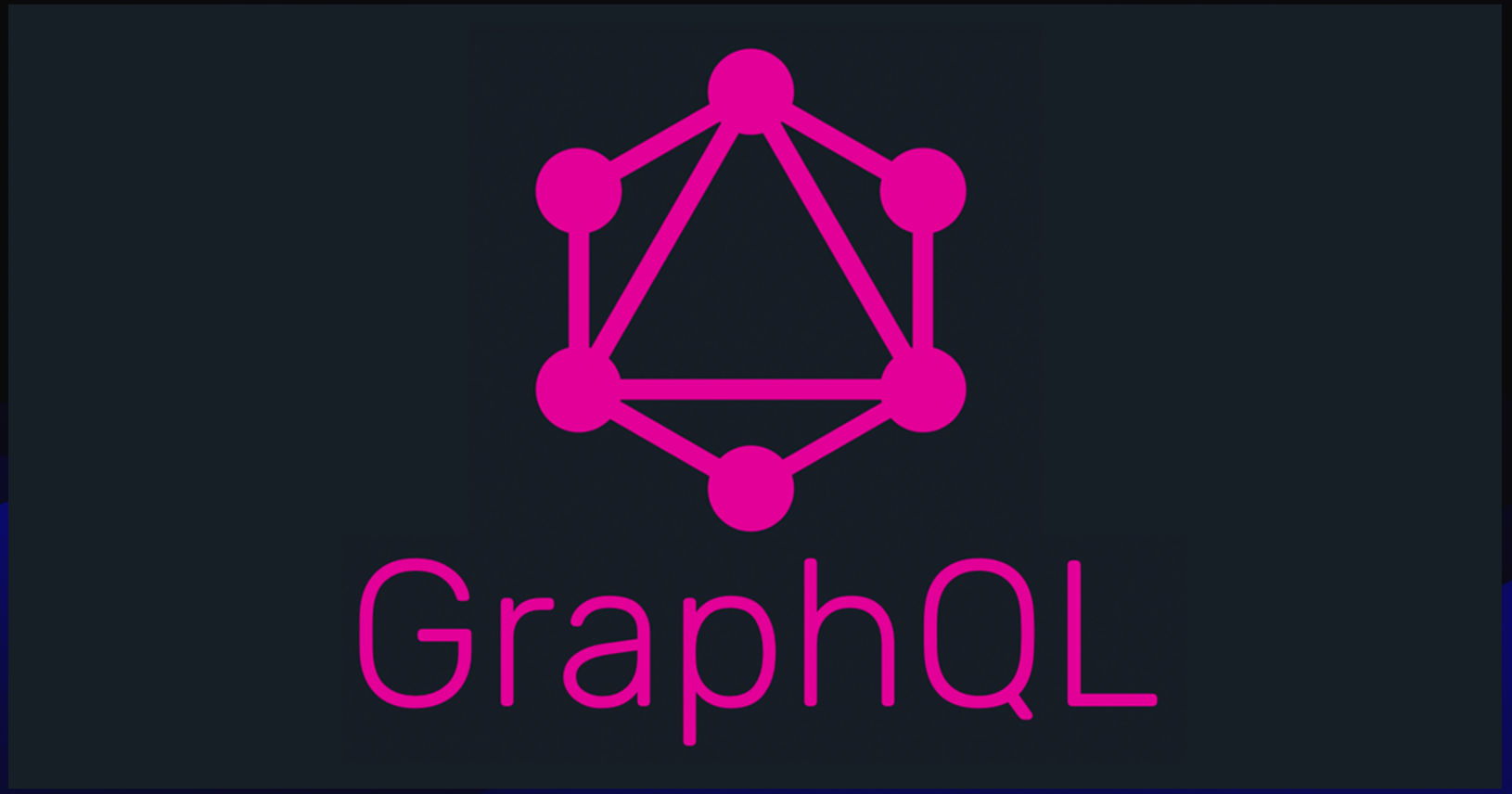
들어본 것 같긴한데...

뭐하는 놈일까...



Query Language

정보를 얻기 위해 보내는 질의문(Query)를 만들기 위한 언어

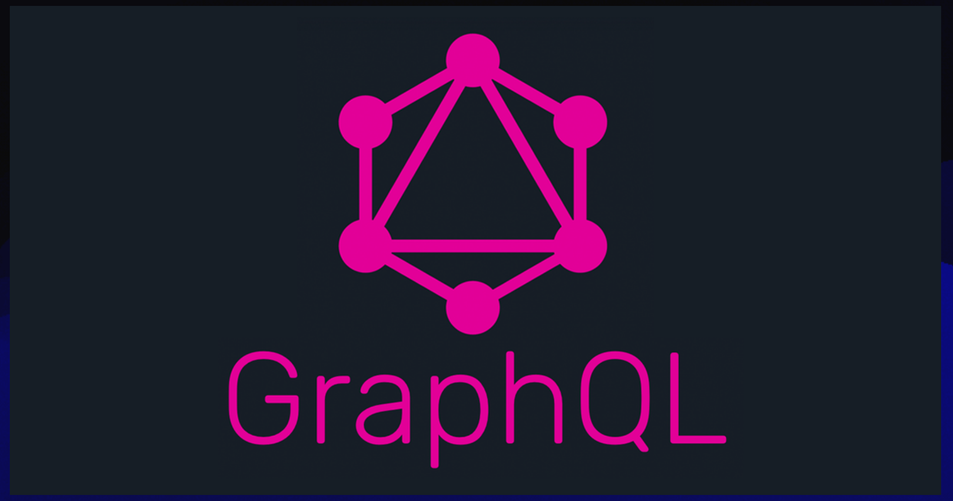


SQL이 서버 -> 데이터베이스 라면

gql은 클라이언트 -> 서버용이다.



VS



클라이언트 -> 서버는 REST API도 있지 않나요?

Create --- POST https://api.younhabooks.com/books

Read --- GET https://api.younhabooks.com/books

Update --- PATCH https://api.younhabooks.com/books/20

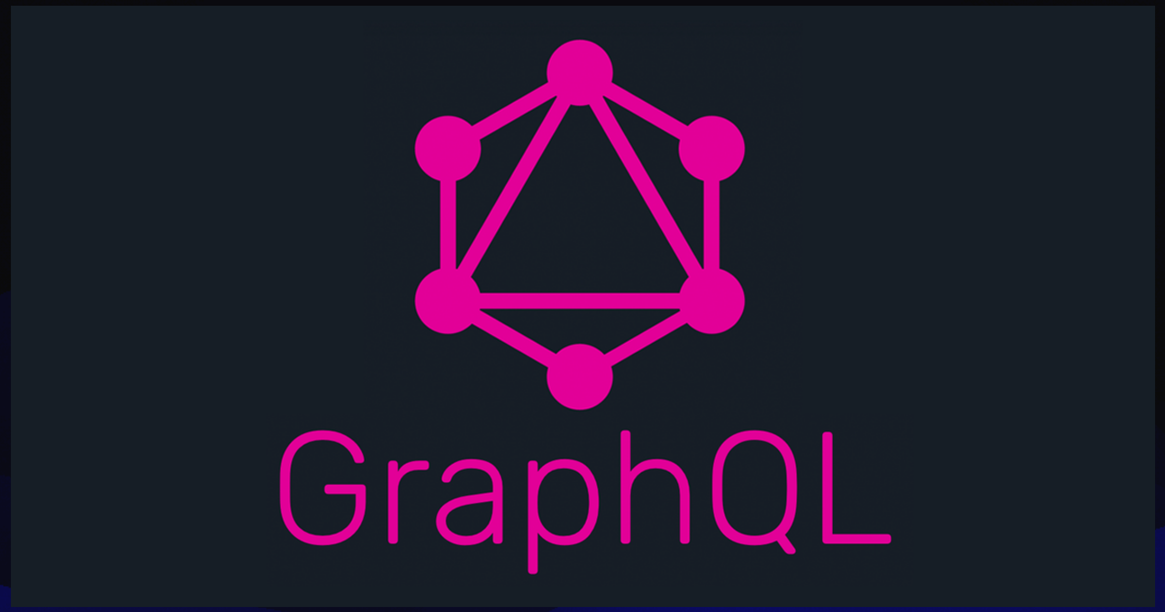
Delete --- DELETE https://api.younhabooks.com/books/101

REST API == 전송에 사용된 HTTP 메소드 + URI로 구성

장점: 요청의 내용이 직관적임

단점: 많은 엔드포인트





POST https://api.younhabooks.com/graphql

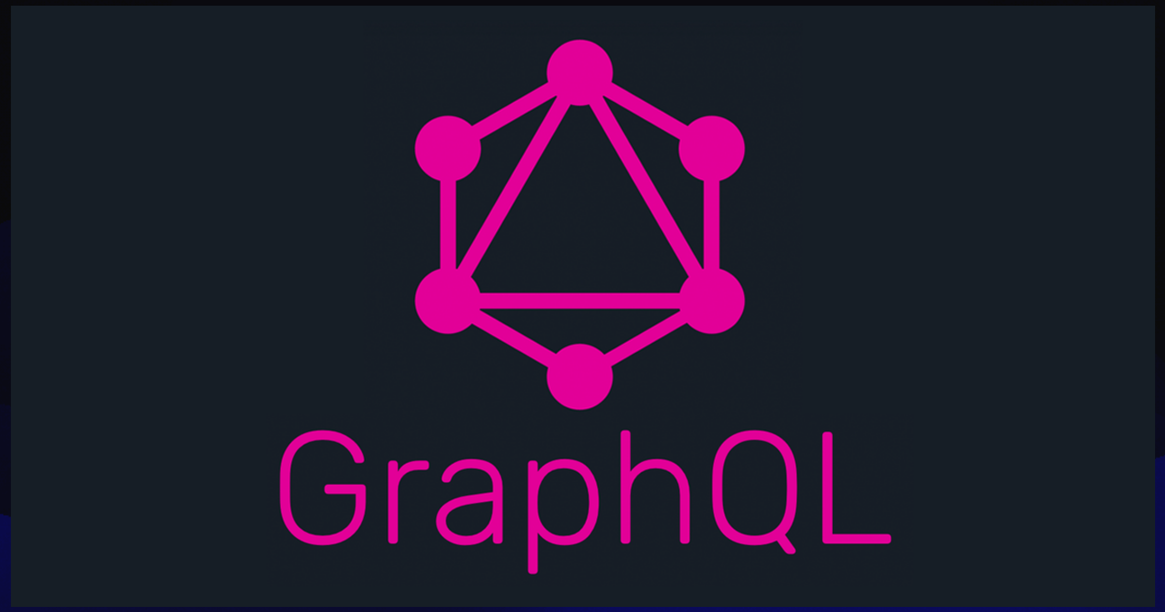
GraphQL == POST 요청 + 하나의 엔드포인트

REST API vs GraphQL

GraphQL은 보통 하나의 엔드포인트를 가진다.

GraphQL은 요청할 때 사용하는 쿼리에 따라 다른 응답을 받을 수 있다.

GraphQL은 원하는 데이터만 받을 수 있다.



GraphQL은 REST API의 다음과 같은 문제를 해결하기 위해 만들어짐

1. Overfetching
2. Underfetching

1. Overfetching

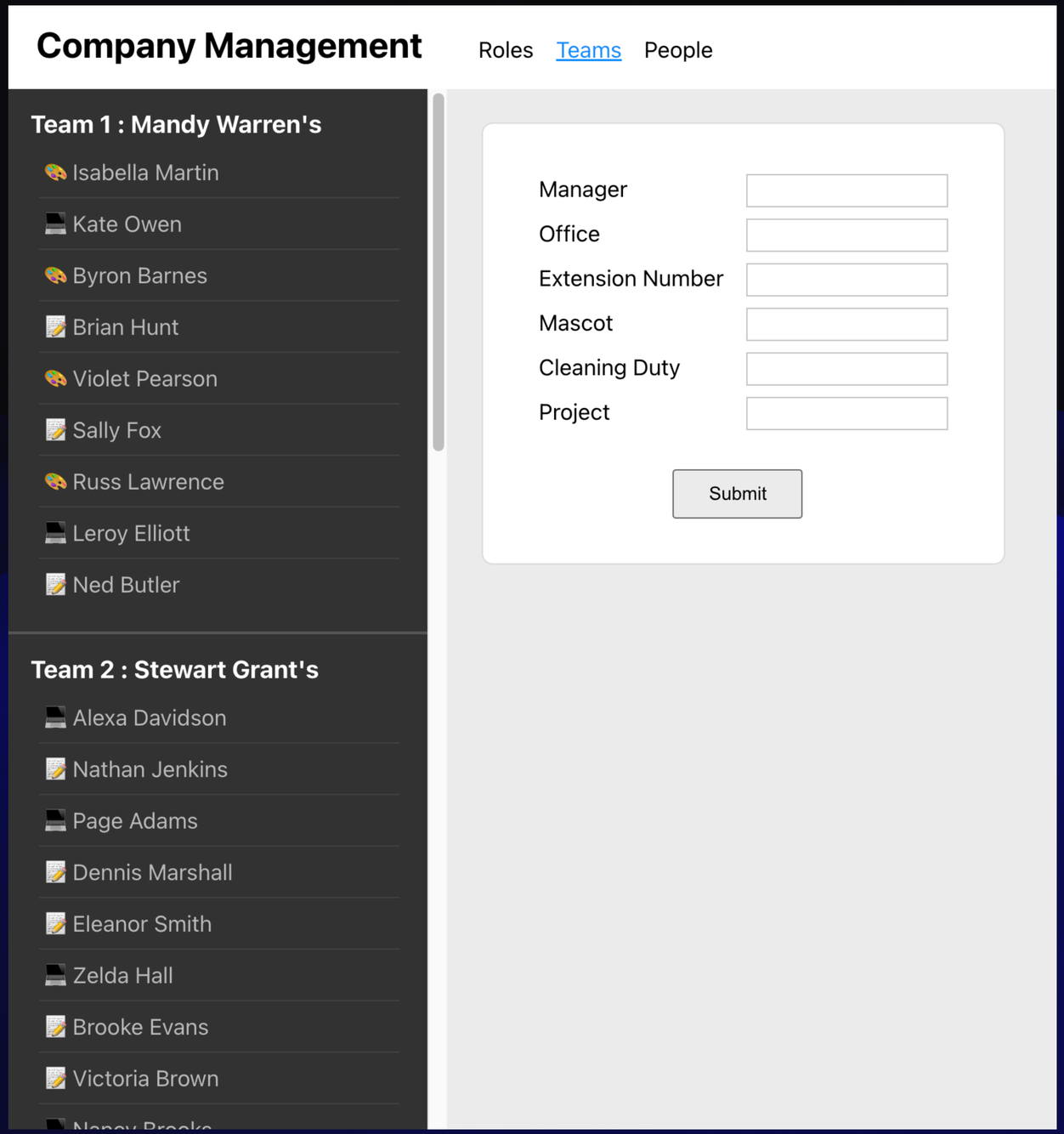
팀, 매니저, 팀에 소속된 사람들을

보여주는 다음과 같은 화면이 있다고 하자.

사이드바를 띄울 때 필요한 정보는

팀의 id, 매니저의 이름

멤버의 이름과 직업일 것이다.



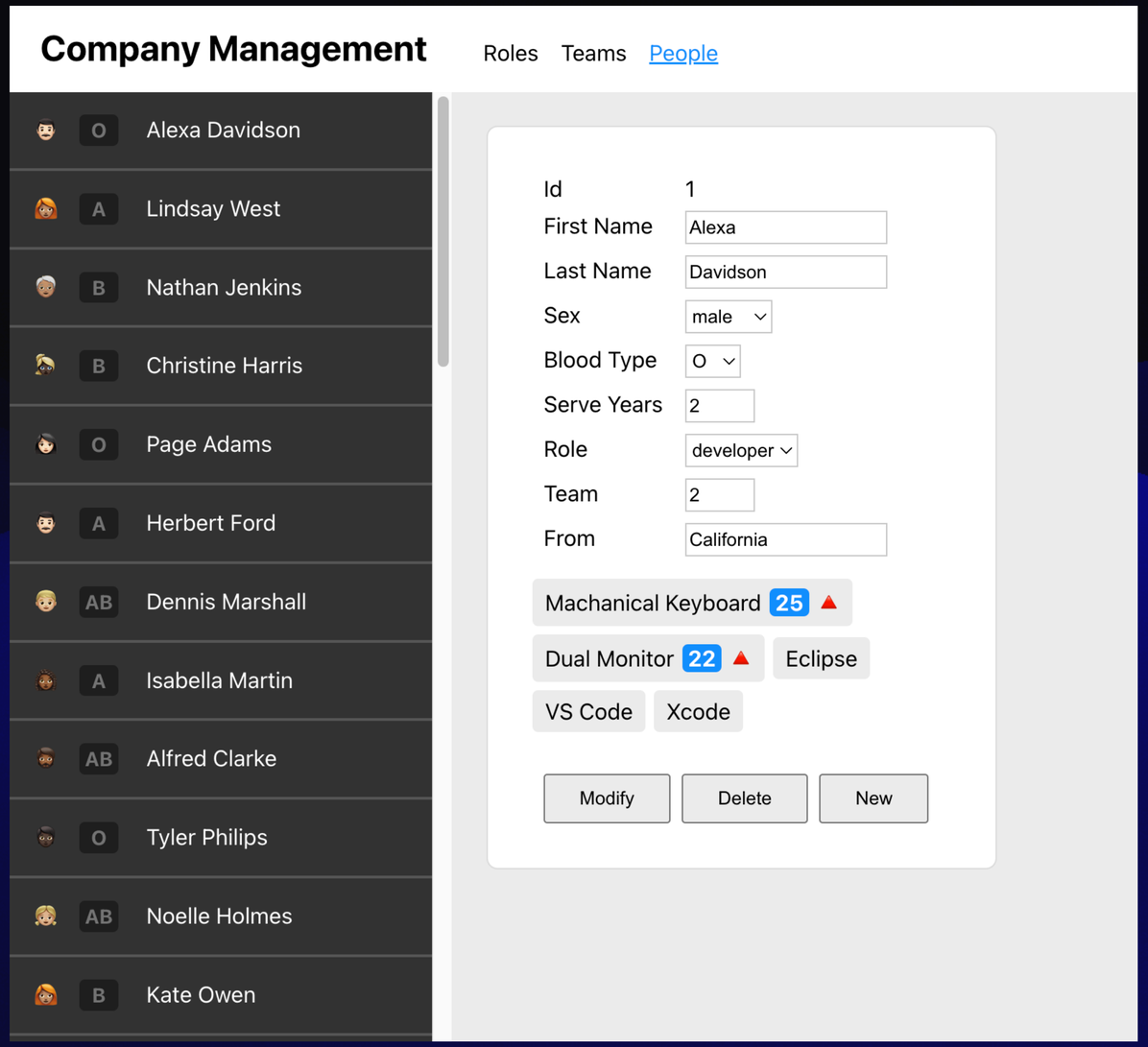
1. Overfetching

그러나 다음과 같은 화면에서는

사이드바를 구현하기 위해서

기존 id, 이름에 더해

성별, 혈액형 정보가 추가로 필요하다



1. Overfetching

만약 RESTful API의 규격대로 만든

API에 요청했다면

멤버에 대한 정보를 요청할 때마다

멤버의 모든 데이터가 반환될 것이다.

필요 이상의 데이터가 전송되는

Overfectching 문제 발생!


```
[
    {
        "id": 1,
        "first_name": "John",
        "last_name": "Doe",
        "role": "developer",
        "sex": "male",
        "blood_type": "A",
    },
    {
        "id": 2,
        "first_name": "Jane",
        "last_name": "Doe",
        "role": "designer",
        "sex": "female",
        "blood_type": "B",
    },
    {
        "id": 3,
        "first_name": "Tom",
        "last_name": "Doe",
        "role": "planner",
        "sex": "male",
        "blood_type": "O",
    }
]
```

1. Overfetching

만약 GraphQL로 요청했다면

POST 메소드로 이와 같은 요청을 보낸다.

query로 멤버의 id, 이름, 역할에 대한 정보를 요청한다.

응답으로는 이와 같은 데이터가 반환된다.

클라이언트가 필요로 하는 정보만 취사 선택해서

서버에 요청할 수 있다!


```
{
  "data": {
    "people": [
      {
        "id": "1",
        "first_name": "Alexa",
        "last_name": "Davidson",
        "role": "developer"
      },
      {
        "id": "2",
        "first_name": "Lindsay",
        "last_name": "West",
        "role": "designer"
      },
      {
        "id": "3",
        "first_name": "Nathan",
        "last_name": "Jenkins",
        "role": "planner"
      },
      {
        "id": "4",
        "first_name": "Christine",
        "last_name": "Harris",
        "role": "developer"
      },
```

2. Underfetching

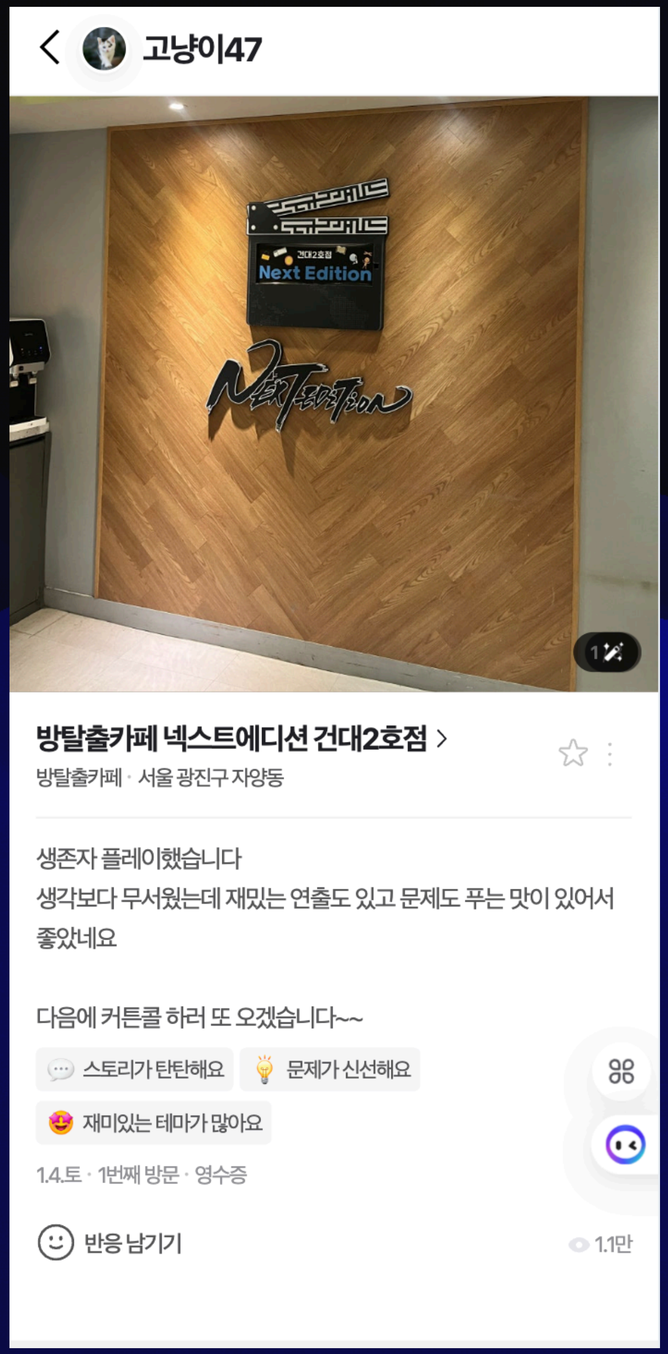
다음과 같은 뷰를 구현해야한다고 생각해보자

1. 매장의 정보
2. 리뷰에 대한 정보
3. 작성자에 대한 정보

RESTful API라면 각각 다른 요청을 통해 받아와야한다.

즉, 하나의 요청만으로 충분한 데이터를 받아오지 못한다!

+ 각 요청별 필요 이상의 데이터도 받아지는 overfetching은 덤



2. Underfetching

```
query {
  team(id: 1) {
    id
    manager
    members {
      id
      first_name
      last_name
      tools {
        id
        used_by
      }
    }
  }
}
```

하지만 GraphQL의

쿼리를 잘 만진다면

계층적인 데이터를

요청할 수 있다.

한 번의 요청으로

필요한 정보만

받아오는 것이다

```
"data": {
  "teams": [
    {
      "id": "1",
      "manager": "Mandy Warren",
      "members": [
        {
          "id": "8",
          "first_name": "Isabella",
          "last_name": "Martin",
          "tools": [
            {
              "id": "pen tablet",
              "used_by": "designer"
            },
            {
              "id": "ergonomic mouse",
              "used_by": "designer"
            },
```

요청에는 mutation!

```
mutatio query {
  editT     team(id: 1) {
    man       id
    cle       manager
  }) {        office
    id        extension_number
    man       mascot
    off       cleaning_duty
    ext       project
    mas     }
    cle   }
    pro
  }
}
```

```
{
  "data": {
    "editTeam": {
      "id": "1",
      "manager": "서해근",
      "office": "101A",
      "extension_number": "#5709",
      "mascot": "Panda",
      "cleaning_duty": "Sunday",
      "project": "Hyperion"
    }
  }
}
```

구현하는 방법?

GraphQL은 요청을 보내기 위한 형식일 뿐

gql로 요청을 보내고 응답을 보내기 위해서는 직접 구현하거나

Apollo와 같이 이미 구현된 라이브러리를 사용하면 된다.

백엔드에서 구현하는 법

Apollo Server는 typeDef와 resolver를 인자로 받아 서버를 생성

```
const server = new ApolloServer({
    typeDefs,
    resolvers,
    cors: {
        origin: '*',
    }
});

server.listen().then(({ url }) => {
    console.log(`🚀  Server ready at ${url}`);
});      You, 13 hours ago • get 함수 제거
```

typeDef

GraphQL 명세에서 사용될 데이터와 요청의 타입

gql(template literal tag) 로 생성

```
const typeDefs = gql`
    type RoleInfo {
        id: ID!
        job: String!
        requirement: String
        members: [People]
        equipments: [Equipment]
        softwares: [Software]
    }
`
```

```
const typeDefs = gql`
    type Query {
        teams(
            manager: String,
            cleaning_duty: String
            ): [Team]
        team(id: ID!): Team
            }
`
```

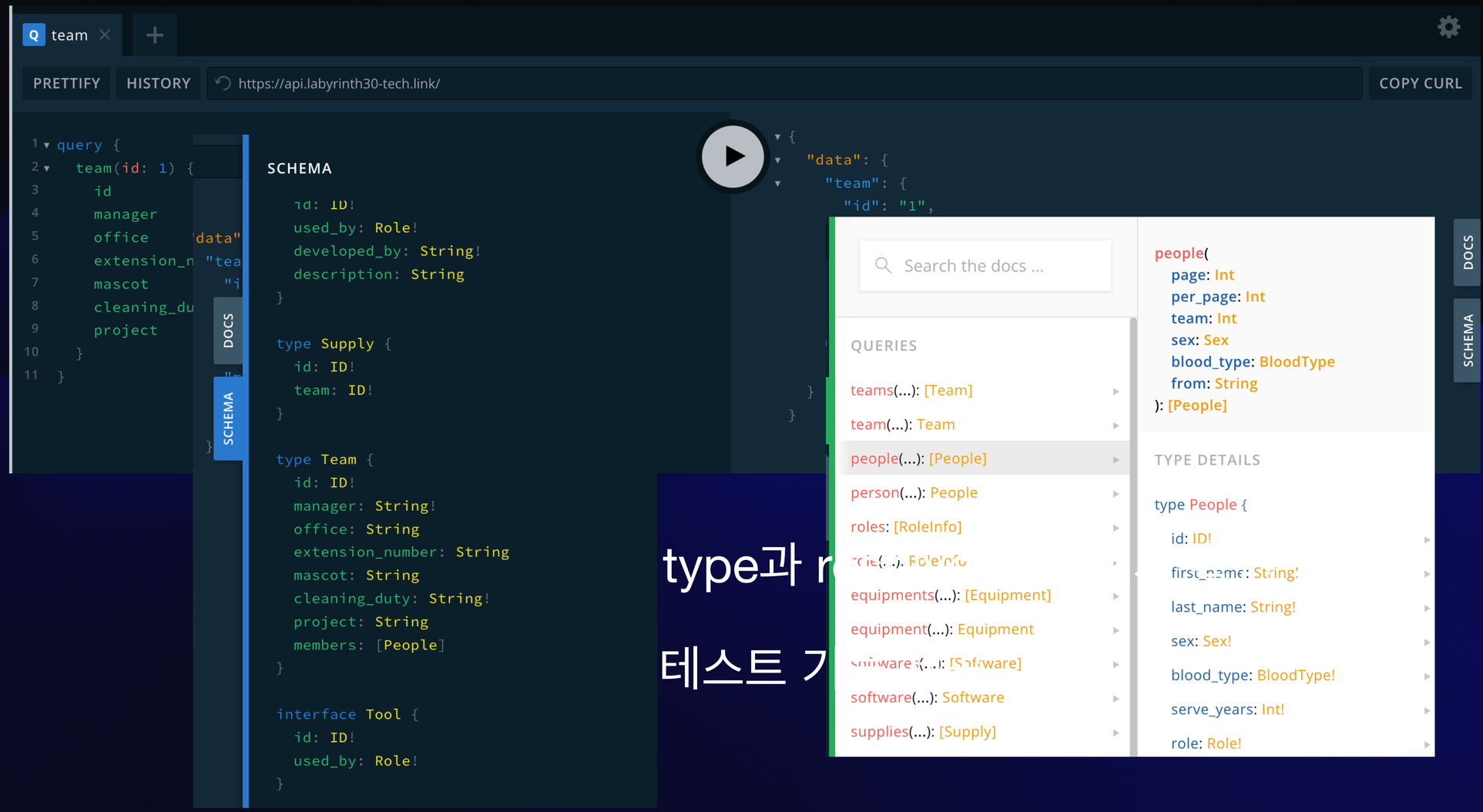
resolver

서비스의 액션들을 함수 형태로 지정

요청에 따라 CRUD를 구현

```
const resolvers = {
    Query: {
        people: (parent, args) => dbWorks.getPeople(args),
        person: (parent, args) => dbWorks.getPeople(args)[0]
    },
    Mutation: {
        postPerson: (parent, args) => dbWorks.postPerson(args),
        editPerson: (parent, args) => dbWorks.editPerson(args),
        deletePerson: (parent, args) => dbWorks.deleteItem('people', args)
    }
}
```

GraphQL Playground



type과 r

테스트 기

프론트에서 구현하는 법

1. gql 쿼리를 작성한다.

2. 렌더링될 컨텐츠id를

저장할 state를 지정

3. 쿼리로부터 데이터를 받아와 렌더링

```
function AsideItems () {
  const roleIcons = {
    developer: '💻',
    designer: '🎨',
    planner: '📝'
  }
  const { loading, error, data } = useQuery(GET_ROLES);
  if (loading) return <p className="loading">Loading ... </p>
  if (error) return <p className="error">Error :(</p>
  return (
    <ul>
      {data.roles.map(({id}) => {
        return (
          <li key={id} className={'roleItem ' +  (contentId === 'id' ? 'on' : '')}
          onClick={() => {setContentId(id)}}>
            <span>{contentId === id ? '🔲' : '⬛'}</span>
            {roleIcons[id]} {id}
          </li>
        )
      })}
    </ul>
  );
}
```

GraphQL의 단점

1. 캐싱

요청들이 복잡한 GraphQL 메세지로 되어 있기 때문에

캐싱하기 용이하지 않다.

2. 성능 이슈

복잡한 쿼리를 해석해서 작업을 실행하기 때문에

REST API에 비해 서버에 부담을 줄 수 있다.

3. 러닝커브

프론트엔드, 백엔드 모두 공부해야 사용할 수 있다.

GraphQL 사용하면 좋은 프로젝트

1. Overfetching 과 Underfetching이 잦은 프로젝트

클라이언트가 데이터 요청에 많은 제어권을 가진 서비스일수록 유리하다.

클라이언트가 중심이 되는 요청이기 때문에

스키마 설계 또한 Client-Centric 하게끔 설계된다.

2. 데이터 업데이트에 대한 실시간 반응이 필요한 서비스

오늘 설명하지 않았지만, '구독' 기능을 사용하면

실시간으로 최신 정보를 나타낼 수 있다.